DEPLOYMENT

giacomo

2026-08-19

Contents

Deploying a finished filter

What happens to `FLX-trimmed-48k.wav` and `FRX-trimmed-48k.wav` after REW-INVERSION.md step 11 has accepted them: how they become BruteFIR coefficients at every sample rate the DAC can be asked for, and how a deployment is made to state what it is made of.

This is the reference. The guide carries a condensed version as its step 12.

---

1. What leaves REW

Exactly two files. Everything below is derived from them, and nothing below ever re-opens REW.

format	WAV, **48 kHz, 32-bit float** — REW's own working rate, so no resampling happens inside REW
length	**131072 samples**, fixed. REW has no tap count; the export length is not selectable
names	`FLX-trimmed-48k.wav`, `FRX-trimmed-48k.wav`
trimmed	yes — `Trim IR to windows` before exporting. It moves the impulse within the file and changes the response by 0.0000 dB

Export with **`Export` → `Export impulse response as WAV`** and REW's default options at 48 kHz. The only two fields worth checking are the sample rate and the bit depth: 32-bit float keeps the coefficients exact, and 48 kHz keeps REW out of the resampling business — SoX does that later, once, with known arithmetic.

Alongside the WAVs, export the **frequency-response TXTs** of the same two filters (`FLX.txt`, `FRX.txt`) with `Smoothing: None`. The deployment does not convolve with them; it uses them as an independent statement of what the filter is supposed to do, and checks the WAVs against it. Without them the declaration in §4 cannot run its TXT↔WAV validation.

2. The four repositories

repo	owns
`../DRC-120.blue` (the *source*)	the measurements, the REW project, the exports, the two filter WAVs — and the **design declaration** that says which file plays which role
`~/devel/open-media-drc` (the *engine*)	public. The scripts, `drc.sh`, BruteFIR, the build. It ships only the generic `flat` set — a dirac pulse, no correction
`~/devel/omdrc-801N` (the *site data*)	this room's `configs/<geometry>/` and `filters/<geometry>/` — the deployed coefficients, their manifests and their sources
`DRC-doc` (here)	the documents

The site split is a recent design change and the important one to understand. The engine is a public project that anybody can install; it has no business carrying one particular room's filters, and one particular room's filters have no business being versioned by the engine's release cycle. So they live apart:

- `open-media-drc` is the *product*, and ships `flat` so that it works out of the box with no correction at all;
- `omdrc-801N` is *this* installation — the Trieste room, the Nautilus 801 — and holds everything room-specific.

Nothing in `omdrc-801N` is secret. It is published as a complete worked example of what a measured, verified deployment actually looks like, which is also why it is worth reading before making your first one.

The two halves are versioned, reviewed and deployed independently, and they are rejoined in three places: CMake's `OMDRC_SITE_DATA_DIRS` at build time, the design scripts' `OMDRC_SITE_ROOT` at publication time, and `make install`, which merges both into `$PREFIX/etc/open-media-drc` so the runtime reads one tree (§7).

That split is also why a deployment **copies** the sources it was given into the site data, under stable role names, with their hashes recorded: an installed machine must not depend on a sibling measurement checkout that will not exist there.

3. Path A — the quick rebuild

For experiments, or when a geometry already exists and only the coefficients changed. No provenance, no tag, no manifest, no green UI status.

```
SITE=~/devel/omdrc-801N
cd ~/devel/open-media-drc
cp ../DRC/DRC-120.blue/FLX-trimmed-48k.wav $SITE/filters/120.blue/rew/
cp ../DRC/DRC-120.blue/FRX-trimmed-48k.wav $SITE/filters/120.blue/rew/

scripts/REW2raw-all-rates.sh \
  -L $SITE/filters/120.blue/rew/FLX-trimmed-48k.wav \
  -R $SITE/filters/120.blue/rew/FRX-trimmed-48k.wav \
  -o $SITE/filters/120.blue

python3 scripts/headroom_calc.py $SITE/filters/120.blue
```

`REW2raw-all-rates.sh` writes `L.raw`, `R.raw` and `sox.txt` into every numeric directory found under the filter root — here `44100/`, `48000/`, `88200/`, `96000/`, `192000/`. Without `-y` it names the existing files and asks before each overwrite.

> **Rate directories: path A discovers, path B creates**
>
> `REW2raw-all-rates.sh` contains no `mkdir`. It enumerates the numeric directories that already exist under `-o DIR` and exits 3 — *"No numeric sample-rate directories found"* — if there are none. **To add a rate on this path you create its directory yourself first**, and its `configs/<geometry>/brutefir-<rate>.conf.in` with it.
>
> Path B is the opposite by design: `deploy_filter.py` creates each rate directory in staging from the manifest's rate list, which comes from `--rates` (default `44100,48000,88200,96000,192000`). The

rule it follows is stated in the provenance document — *do not discover rates only from directories that happen to exist* — because a rate that silently vanished along with its directory is exactly the failure a manifest exists to prevent.

Then set `attenuation:` in each `configs/120.blue/brutefir-<rate>.conf.in` to what `headroom_calc.py` printed, and rebuild.

Edit the `.conf.in` template, never the `.conf`. `configs/*/*.conf` is gitignored and generated: CMake renders each template at install time, rewriting `@REPO_DIR@` to the installed site directory. A `.conf` you edit by hand is overwritten on the next install, and was never the file git tracks.

`filters/<geometry>/rew/` holds the REW-exported WAVs. BruteFIR never reads them; only `REW2raw` does.

---

4. Path B — the declared, tagged deployment

The real path. It produces a bundle whose identity is a hash of everything that went into it, which is what lets the web UI show a filter's provenance in green instead of grey.

4.1 Declare the roles — in the source repository

A filename is not evidence. `L.txt` is not proof that a file holds the left measurement, and renaming things to fit a convention makes that worse, not better. So the designer states the roles once, explicitly, and the tool records what each chosen file actually contains.

Discovery first, read-only — it finds the newest `.mdat` by modification time, locates its sibling `<stem>.txts`, ranks the compatible L/R pairs, and prints a complete candidate command. It never opens the project:

```
python3 scripts/declare_filter_design.py \
  --suggest-from-source-root ../DRC/DRC-120.blue
```

Then the declaration itself, dry-run first (no `--write`):

```
python3 scripts/declare_filter_design.py \
  --source-root ../DRC/DRC-120.blue \
  --geometry 120.blue --design-id rscreen-fdw8-20260813 \
  --description "120.blue Rscreen, 8-cycle FDW correction" \
  --measurement-left  "120.blue.Rscreen.txts/L 120.Rscreen.orig.txt" \
  --measurement-right "120.blue.Rscreen.txts/R 120.Rscreen.orig.txt" \
  --measurement-sum    120.blue.Rscreen.txts/LR.orig.txt \
  --filter-left-txt    120.blue.Rscreen.txts/FLX.txt \
  --filter-right-txt   120.blue.Rscreen.txts/FRX.txt \
  --filter-left-wav    FLX-trimmed-48k.wav \
  --filter-right-wav   FRX-trimmed-48k.wav \
  --corrected-left-txt 120.blue.Rscreen.txts/L.Filtered.txt \
  --corrected-right-txt 120.blue.Rscreen.txts/R.Filtered.txt \
  --corrected-sum-txt  120.blue.Rscreen.txts/LR.Filtered.txt \
  --sum-mode vector_average
```

What it checks, in eight stages: argument roles, Git context, hashing and header parsing, measurement consistency (same frequency grid, a common acoustic timing reference, the declared room/setup markers), **TXT↔WAV alignment**, provenance assembly, response prediction, and the write preview.

The TXT↔WAV check is the interesting one. It fits exactly two transformations between the filter TXT and the filter WAV — one integer causal delay (8192 samples, from the trim) and one constant export gain (−3.0003 dB) — and then requires the *residual* amplitude and phase error to be small. A frequency-dependent discrepancy is rejected. This catches the whole class of "the WAV I exported is not the filter I designed" errors.

`--sum-mode` states what the aggregate export means, and it is checked against the TXT header rather than believed. **For a filter built by this guide's procedure it is `vector_average`**: step 3c forms the vector `L + R` and then subtracts 6.0206 dB, which is `(L + R) / 2`.

The output path is derived, not chosen: `omdrc-designs/<geometry>/<design-id>/design.json`.

A dirty source checkout is allowed here, because the new exports and the declaration that names them want to be committed together.

4.2 Commit and tag — still in the source repository

```
git -C ../DRC/DRC-120.blue add -- \
  omdrc-designs/120.blue/rscreen-fdw8-20260813/design.json \
  <every file named as a role above>
git -C ../DRC/DRC-120.blue commit -m "Declare 120.blue Rscreen FDW8 filter"
git -C ../DRC/DRC-120.blue tag -a 120.blue-rscreen-fdw8-20260813 \
  -m "120.blue Rscreen FDW8 correction"
```

The annotated tag is the trust anchor. An annotated tag is an immutable named Git object; a *signed* annotated tag additionally establishes who made it. Deployment requires one by default — `--allow-commit-ref` exists as an explicit lower-assurance exception, and says so in the manifest.

The `.mdat` is deliberately not part of this. `--project` would add its path and hash as archival evidence only; the declaration plus the annotated tag are the intended anchor, and nothing in the chain ever opens the REW project.

4.3 Build and publish — from the engine checkout, into the site repo

```
cd ~/devel/open-media-drc
export OMDRC_SITE_ROOT=~/devel/omdrc-801N      # or pass --site-root
python3 scripts/new_filter_design.py \
  --source-root ../DRC/DRC-120.blue \
  --source-ref 120.blue-rscreen-fdw8-20260813 \
  --declaration omdrc-designs/120.blue/rscreen-fdw8-20260813/design.json
```

Dry run unless `--write` is given: it performs the complete build and changes no repository file. Review every `PASS`, then repeat with `--write`.

Useful options: `--rates` (default `44100,48000,88200,96000,192000`), `--safety-margin` (default 1.0 dB), `--attenuation` (default `auto`), `--replace-design`.

The eight-step transaction, which is what makes this worth doing:

1. **Resolve and verify.** Every input must be a regular file inside the source repository — no symlinks, no duplicate roles. `HEAD` must equal the annotated tag's target. The tag object, source commit, declaration blob and every SHA-256 are recorded and checked. A selected file that is untracked, dirty, absent from the tag or different from its declared hash **stops the deployment** — unlike at declaration time. Unrelated untracked files do not.
2. **Parse the TXT headers.** Declared L/R roles, a common acoustic timing reference, the same frequency grid. Smoothing is recorded and shown in the UI rather than hidden.
3. **Re-check filter TXT against filter WAV** in complex frequency space, as in 4.1, storing both fitted transformations and the residuals in the manifest.
4. **Build in fresh staging** — canonical source copies, every *explicitly requested* rate, headroom, graph data, candidate manifest. Rates come from `--rates`, never from whichever directories happen to exist.
5. **Re-read and hash every staged artifact.** Each candidate BruteFIR config is parsed and its rate, format, two coefficient paths and attenuation must match the manifest, with the configured attenuation at least the calculated requirement.
6. **Compute the corrected response** by complex multiplication, and where independent corrected L / R / L+R exports were declared, compare against them and reject differences beyond the declared RMS magnitude and phase limits.
7. **Publish atomically**, only after every mandatory check passes. Never half an L/R pair, never one rate at a time.
8. **Verify** with `verify_filter_bundle.py`.

During step 4 the console prints, per rate, the exact FIR coefficient scale and the signed SoX gain in dB (magenta), then the worst L/R FFT peak and the safety-margin arithmetic (yellow), then the config bake and read-back (blue).

4.4 Verify

```
python3 scripts/verify_filter_bundle.py --all --require-sources \
        --site-root ~/devel/omdrc-801N
```

Read-only. Checks the bundle ID, the source copies, the graph dependencies, the configs, the **exact runtime RAW hashes** and the headroom. CMake runs it with `--no-next` during configure.

4.5 Commit the site data, install, select

The commit belongs to **omdrc-801N**, not to the engine:

```
git -C ~/devel/omdrc-801N add -- filters/120.blue configs/120.blue
git -C ~/devel/omdrc-801N commit -m "Deploy 120.blue rscreen-fdw8-20260813"
git -C ~/devel/omdrc-801N push
```

Optionally tag that site commit with the geometry and short bundle ID, as the deployment-side release anchor. Then build and select, in the engine checkout:

```
cd ~/devel/open-media-drc/build
cmake .. -C ../host.cmake && make && sudo make install
./drc.sh design --list
./drc.sh design @rscreen-fdw8-20260813
```

On a separate playback box the first half is just `git pull` in `omdrc-801N`, then the same build and install.

Every command above prints its own **NEXT** block: the remaining steps with a per-step working directory. That is worth reading rather than skimming, because the source commit, the site commit and the build happen in three different repositories, and the one thing that reliably goes wrong is running the right command in the wrong one.

---

5. The resampling arithmetic

`REW2raw.sh` converts one REW WAV into one BruteFIR `FLOAT64_LE` raw file:

```
scripts/REW2raw.sh [--exact-output] [--no-keep-intermediate] \
                   [--intermediate-dir DIR] in.wav [out.raw] [wav|raw] [rate]
```

SoX does the rate conversion at `rate -v -L -s` through a float64 intermediate, which is kept for inspection unless `--no-keep-intermediate`.

There is no peak normalisation, and this is deliberate. A sampled impulse response represents `T·h(nT)`, where `T` is the sampling period. Change the rate and the coefficients must be rescaled by the ratio of the periods:

```
scale   = T_target / T_source = Fs_source / Fs_target
gain_dB = 20·log10(scale)
```

So the 48 kHz export becomes the 192 kHz coefficient set at `48000/192000` = 0.25, i.e. **−12.04 dB** — not because anything is being made quieter, but because four times as many coefficients now sample the same continuous impulse. Peak values in `sox.txt` are diagnostics, not targets. (J.O. Smith, *Physical Audio Signal Processing*, "Sampling the Impulse Response".)

`sox.txt` in each rate directory records the source WAVs, the exact command lines, the applied gain and the measured statistics.

---

6. Headroom

```
python3 scripts/headroom_calc.py [filter_root] [--variant V] [--format F]
                                 [--margin dB] [--json]
```

BruteFIR convolves input audio bounded by ±1.0 with `h[n]`. At the frequency of maximum gain, a full-scale sine clips if `|H(f)| > 1`. So:

```
attenuation_dB = max(0, peak_gain_dB + safety_margin_dB)
```

`peak_gain_dB` is the maximum magnitude of the FFT of the impulse response — the FFT bins *are* `H(f)`. The default safety margin is 1 dB, and the result is rounded up to 0.1 dB.

Run it after every filter regeneration. Path B does it automatically and refuses to publish a config whose `attenuation:` is below the requirement; path A does not, and a filter with net boost anywhere will clip without it.

---

7. The site split, concretely

`configs/<geometry>` and `filters/<geometry>` are read and written through a *site root*, resolved in this order:

1. `--site-root DIR` — accepted by `new_filter_design.py`, `deploy_filter.py` and `verify_filter_bundle.py`;
2. `OMDRC_SITE_ROOT`;
3. the engine checkout — the original single-repository layout, still the default, and no longer what this installation uses.

Here the answer is always `~/devel/omdrc-801N`.

At build time, point the engine's CMake at it. The search path is ordered and first match wins, so the engine keeps supplying `flat` while the site repo supplies `120.blue`:

```
# in the engine checkout's host.cmake, where box-specific values belong
set(OMDRC_SITE_DATA_DIRS "${CMAKE_SOURCE_DIR};$ENV{HOME}/devel/omdrc-801N"
    CACHE STRING "Search path for configs/<geo> + filters/<geo>")
```

or, once, on the command line:

```
cmake -S . -B build \
  -DOMDRC_SITE_DATA_DIRS="$PWD;$HOME/devel/omdrc-801N" \
  -DGEOMETRY=flat -DGEOMETRIES=120.blue
sudo cmake --install build
```

A geometry that no search directory defines is skipped with a warning rather than failing the configure, so one missing set never blocks the others.

At publication time, the design tooling reads the same split:

```
export OMDRC_SITE_ROOT=~/devel/omdrc-801N
python3 scripts/new_filter_design.py --source-root ... --source-ref ... \
        --declaration ... --write
```

At verification time it takes `--site-root` explicitly:

```
python3 scripts/verify_filter_bundle.py --all --require-sources \
        --site-root ~/devel/omdrc-801N
```

Design on one machine, play back on another, is then just git: publish and commit in `omdrc-801N` on the design box, pull on the playback box, reinstall.

Three variables are involved and they are **not** the same thing:

variable	used by	means
`OMDRC_SITE_DATA_DIRS`	CMake	semicolon-separated search path (first match wins) for the *sources* of `configs/<geo>` + `filters/<geo>`
`OMDRC_SITE_ROOT`	the design scripts	the one checkout they read and write
`OMDRC_SITE_DIR`	`drc.sh` at runtime	room data in the *installed* `etc/open-media-drc`

`OMDRC_SITE_DIR` predates the split and is unrelated to it — except when running `drc.sh` **uninstalled** from the engine checkout, where it defaults to that checkout and so finds only `flat`. Point it at the site repo:

```
OMDRC_SITE_DIR=~/devel/omdrc-801N ./drc.sh 120.blue 48000
```

It takes a single directory, not a search path, so uninstalled `drc.sh` sees either the engine's `flat` or the room sets — not both at once. An installed system is unaffected: `make install` merges both halves into `$PREFIX/etc/open-media-drc` and `drc.sh` reads only that.

---

8. What the deployed tree looks like

This is `~/devel/omdrc-801N` as it actually stands — `120.blue` with the 2025 measurements as the `default` design and a verified `@rscreen-20260812` alongside it:

```
configs/120.blue/
  brutefir-<rate>.conf.in                  template, tracked
  brutefir-<rate>@rscreen-20260812.conf.in
  brutefir-<rate>.conf                     rendered at install, gitignored
filters/120.blue/
  <rate>/L.raw  <rate>/R.raw               the default design
  <rate>/@rscreen-20260812/L.raw           an immutable A/B design
  <rate>/sox.txt                           the conversion log
  provenance/<design>.json                 hash-bound manifest
  provenance/<design>.source.json          the build recipe it was made from
  analysis/<design>.json                   precomputed response curves
  source/<design>/                         role-named copies of the declared inputs
    measurement-L.txt  measurement-R.txt  measurement-L+R.txt
    filter-L.txt  filter-R.txt  filter-L.wav  filter-R.wav
    corrected-L-independent.txt  corrected-R-independent.txt
    corrected-L+R-independent.txt
  rew/FLX-trimmed-48k.wav  rew/FRX-trimmed-48k.wav
```

for `<rate>` in 44100, 48000, 88200, 96000, 192000.

Two details worth reading off that tree. The `default` design's `source/` directory has **no `.wav` files and no corrected exports** — it predates the declaration workflow, and that is exactly the difference a manifest makes visible. And `@rscreen-20260812` carries all three `corrected-*-independent` exports, which is what let step 6 of the transaction check the *predicted* corrected response against REW's own.

Inside a config template the coefficient path is absolute-by-substitution:

```
coeff "c-l" {
        filename: "@REPO_DIR@/filters/120.blue/48000/@rscreen-20260812/L.raw";
        format: "FLOAT64_LE";
        attenuation: 0.0;
};
```

`@REPO_DIR@` is rewritten by CMake at install time. That is the whole reason the `.conf` is generated rather than tracked: the same template has to work from the checkout, from `/usr/local/etc`, and from whatever prefix a different machine installs into.

Geometry and design stay separate concepts. `120.blue` is where the speakers are; `@rscreen-20260812` is which correction is loaded into it. `./drc.sh design --list` and `./drc.sh design @<id>` switch between them and the web control exposes the same selector — which makes an A/B between two filters a one-line change rather than a rebuild.

The `bundle_id` is the SHA-256 of a canonical identity covering the whole source-provenance object (repository, commit, annotated tag object, declaration, lineage, attestation), every source artifact hash, the runtime config and RAW hashes and settings, and the analysis hash. Editing any provenance the UI displays therefore **invalidates the bundle** rather than quietly changing a label. The response page releases its stored room curves only when the running coefficients still match the manifest exactly.

---

9. Script index

script	role in a deployment
`declare_filter_design.py`	assign roles, hash inputs, validate TXT↔WAV, write `design.json`. Never starts REW
`filter_design_suggest.py`	read-only discovery behind `--suggest-from-source-root`
`new_filter_design.py`	the whole build from an annotated tag: verify, stage, publish
`deploy_filter.py`	the lower-level builder `new_filter_design.py` drives
`verify_filter_bundle.py`	read-only verification of a published bundle
`filter_workflow_next.py`	the shared **NEXT** operator handoff. Not called directly
`REW2raw.sh`	one WAV → one raw, with the `Fs_source/Fs_target` scale
`REW2raw-all-rates.sh`	the L/R pair for every existing numeric rate directory
`headroom_calc.py`	required BruteFIR `attenuation:` from the worst-case FFT gain
`rew_mdat_audit.py`	optional archival evidence through the REW API. **Not** a deployment dependency

10. Checklist

Leaving REW

- ☐ step 11 of the guide passed on both channels
- ☐ `FLX-trimmed-48k.wav` / `FRX-trimmed-48k.wav`, 48 kHz, 32-bit float
- ☐ `FLX.txt` / `FRX.txt` exported with `Smoothing: None`
- ☐ the measurement exports the declaration will name are the ones the filter was actually built from

Declaring

- ☐ dry run reviewed — every parsed header is the file you meant
- ☐ `--sum-mode vector_average` for a filter built by this procedure
- ☐ declaration **and** every named input committed together
- ☐ annotated tag created at that commit

Deploying

- ☐ dry run reviewed, every `PASS` read
- ☐ `--write`, then `verify_filter_bundle.py --all --require-sources`
- ☐ `OMDRC_SITE_ROOT` (or `--site-root`) pointed at `~/devel/omdrc-801N`
- ☐ site data committed **in `omdrc-801N`**, not in the engine checkout
- ☐ `cmake && make && sudo make install`
- ☐ `./drc.sh design @<design-id>`, and the UI shows the expected tag, source commit and bundle ID